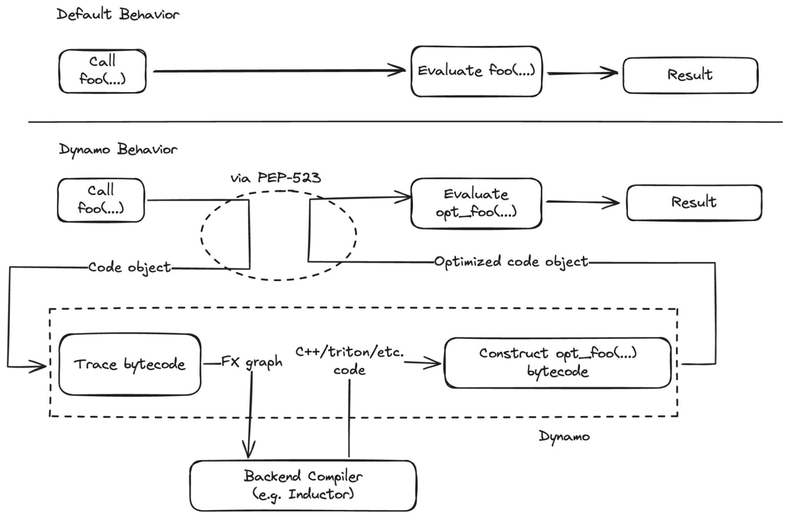
PyTorch CONFERENCE EUROPE 2026

TorchDynamo Debugging Tools for Power Users

William Wen (williamwen@meta.com), Animesh Jain (anijain@meta.com)

What is Dynamo?

Dynamo is the graph capture mechanism of **`torch.compile`**. It uses **PEP 523**, CPython's custom frame evaluation hook, to intercept every Python function call. Dynamo then **symbolically interprets** the Python bytecode to **extract an FX graph** of PyTorch ops, without running tensor compute. It then sends the FX graph to the **backend compiler** for optimization, and finally **returns Python bytecode** to run the compiled graph, which is **run in place** of the original function.



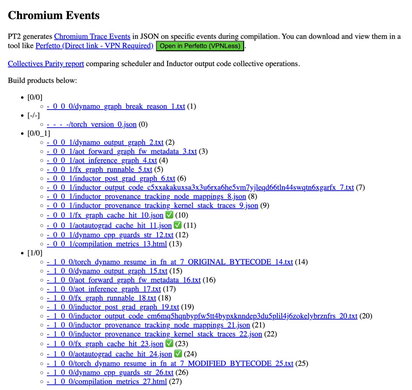
Comparison of execution behavior between eager/default and **`torch.compile`**/Dynamo

Check out the **`torch.compile`** programming model docs for more insight into Dynamo internals!

Logging

Use **`TORCH_TRACE`**/**`tlparse`** to produce a browser-viewable compilation report. It is useful for extracting **`torch.compile`** artifacts from large models. To view detailed **`torch.compile`** logs, especially on smaller models, you can use **`TORCH_LOGS`**.

```python
@torch.compile
def fn(x):
    y = x + 1
    print("debug:", y.shape)  # graph break
    return y * 2

fn(torch.ones(3, 3))
```




```
$ TORCH_TRACE=/tmp/trace python graph_break_example.py
$ tlparse /tmp/trace/
```

```
$ TORCH_LOGS="graph_breaks" python graph_break_example.py
Graph break in user code at graph_break_example.py:7
Graph Break Reason: Encountered graph break when attempting to trace CALL: a function call, e.g. f(x, y):
```

Compile-time Breakpoints

Use **`comptime.breakpoint()`** to pause Dynamo tracing in a user function. You can then inspect Dynamo's internal state - for example, you can see how Dynamo is symbolically representing each variable.

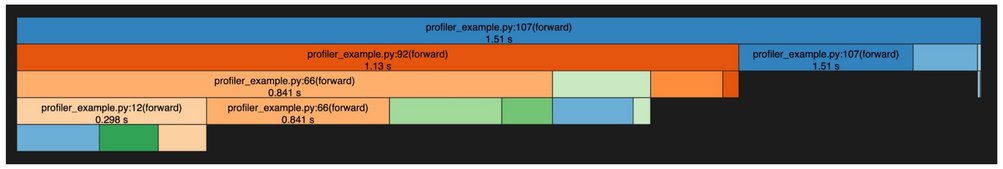
```python
@torch.compile
def fn(x, y, mode="add"):
    if mode == "add":
        z = x + y
    else:
        z = x * y
    scale = z.shape[0]
    comptime.breakpoint()
    return z.relu() * scale
```

```
(Pdb) ctx.print_locals()
x = FakeTensor(..., size=(3, 4))
y = FakeTensor(..., size=(3, 4))
mode = 'add'
z = FakeTensor(..., size=(3, 4))
scale = 3

(Pdb) ctx.get_local("mode").as_python_constant()
'add'  # extract symbolic variables and interact with them

(Pdb) ctx.get_local("mode").python_type()
<class 'str'>

(Pdb) ctx.print_graph()  # print the currently traced graph
def forward(self, L_x_: "f32[3, 4]", L_y_: "f32[3, 4]"):
    l_x_ = L_x_
    l_y_ = L_y_
    z: "f32[3, 4]" = l_x_ + l_y_
```

Compile Time Profiler

Use **`TORCH_COMPILE_DYNAMO_PROFILER`** to profile Dynamo compile time on each traced function. You can use tools like **`snakeviz`** to see which functions Dynamo is spending the most time tracing.

```
TORCH_COMPILE_DYNAMO_PROFILER=1 python script.py          # print to stdout
TORCH_COMPILE_DYNAMO_PROFILER=/tmp/dynamo.prof python script.py  # save for snakeviz
```



Bytecode Debugger

Use **`bytecode_debugger.debug()`** to run Dynamo-generated bytecode step-by-step, like **`pdb`**. At each bytecode instruction, you can inspect the CPython stack. This tool is helpful for developers in fixing incorrect bytecode.

```python
@torch.compile
def fn(x, d):
    y = x + 1
    d["result"] = y.sum()
    return y

with torch._dynamo.bytecode_debugger.debug():
    fn(torch.randn(4), {})
```

```
(bdb) l 16, 30                # list instructions 16-30
    16 [104]: CALL 1                      # call compiled graph
    17 [112]: STORE_FAST graph_out_0      # store compiled graph result
    ...
    22 [124]: LOAD_FAST d                 ┐
    23 [126]: LOAD_ATTR update            │
    24 [146]: LOAD_CONST result           ├ # side effect replay:
    25 [148]: LOAD_FAST graph_out_0       │ # d["result"] = compiled_result
    26 [150]: LOAD_CONST 1                │
    27 [152]: BINARY_SUBSCR               │
    28 [156]: BUILD_MAP 1                 │
    29 [158]: CALL 1                      ┘
```

```
(bdb) s 17                # Step to  instruction 18
(bdb) locals
  x = tensor([ 1.07,  0.09,  0.99, -0.92])
  d = {}
  graph_out_0 = [tensor([2.07, 1.09, 1.99, 0.08]),
                 tensor(5.23)]

(bdb) s 11                # Step to  instructions 29
(bdb) stack               # about to call d.update({"result": ...})
  [0] []
  [1] tensor([2.07, 1.09, 1.99, 0.08])
  [2] <method 'update' of dict>
  [3] {}
  [4] {'result': tensor(5.23)}
```